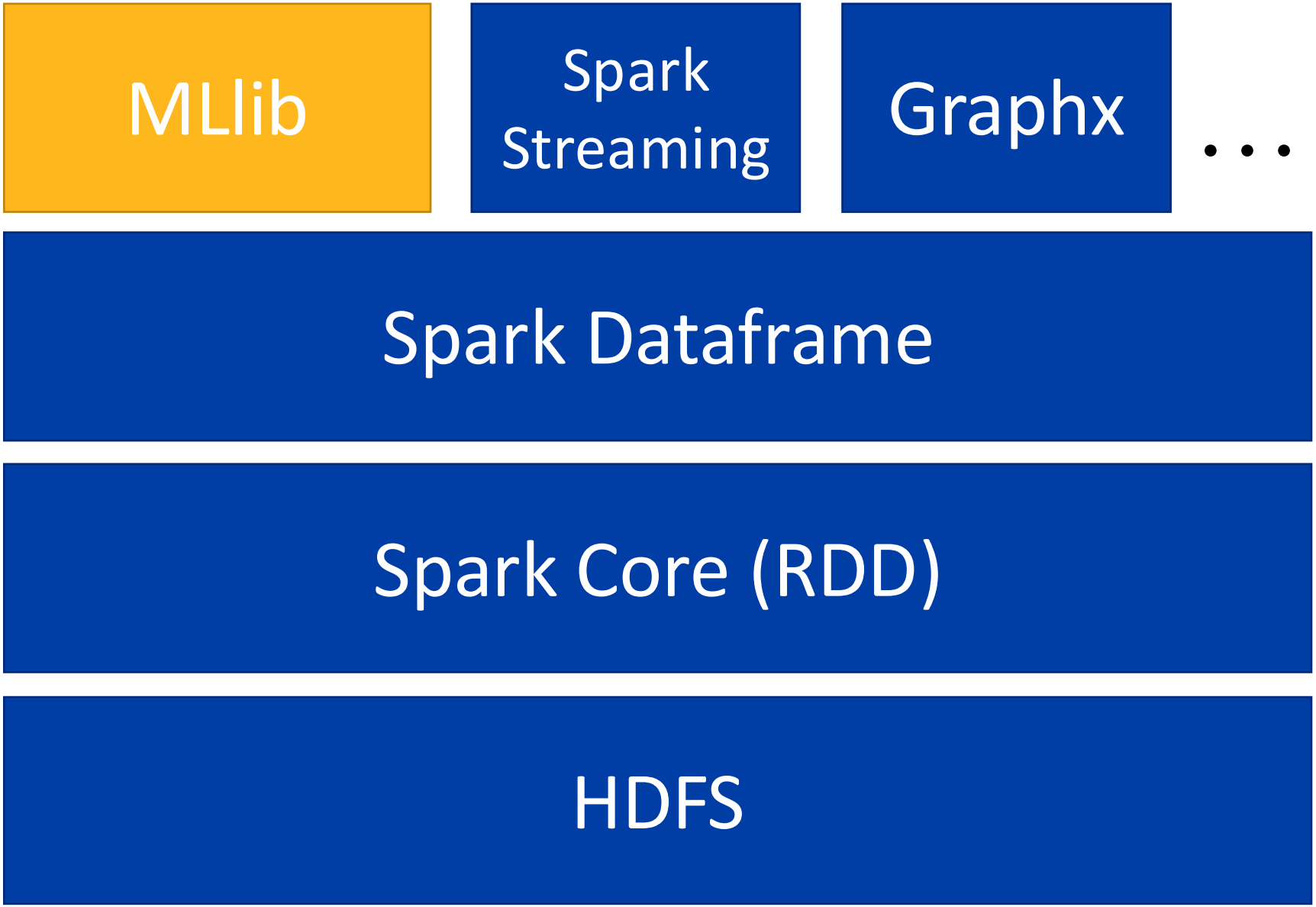
Big Data Meets Machine Learning

Apache Spark MLlib

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

MLlib


MLlib
Spark Streaming
Graphx
. . .
Spark Dataframe
Spark Core (RDD)
HDFS

Machine Learning Algorithms

- Supervised learning
  - Given a set of features and labels
  - Builds a model that predicts the label from the features
  - E.g., classification and regression
- Unsupervised learning
  - Given a set of features without labels
  - Finds interesting patterns or underlying structure
  - E.g., clustering and association mining

Overview of MLlib

- Simple primitives
- Basic Statistics
- Extractors, transformations
- Estimators
- Evaluators
- Model tuning

spark.mllib Vs spark.ml

- spark.mllib
  - RDD-based library which is now in maintenance mode
  - ~~Will be deprecated in Spark 3.x~~
  - Not recommended to use
- spark.ml
  - Dataframe-based API
  - Recommended
  - Replaces (almost) everything in the RDD-API
- Be aware when searching online on which API is used

Simple Primitives

- Local Vector (Data Type)
  - To represent features
  - Example: (1.2, 0.0, 0.0, 3.4)
  - Dense vector [1.2, 0.0, 0.0, 3.4]
  - Sparse vector [0, 3], [1.2, 3.4]

$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$$

- Local Matrix (Data Type)
  - Dense and Sparse

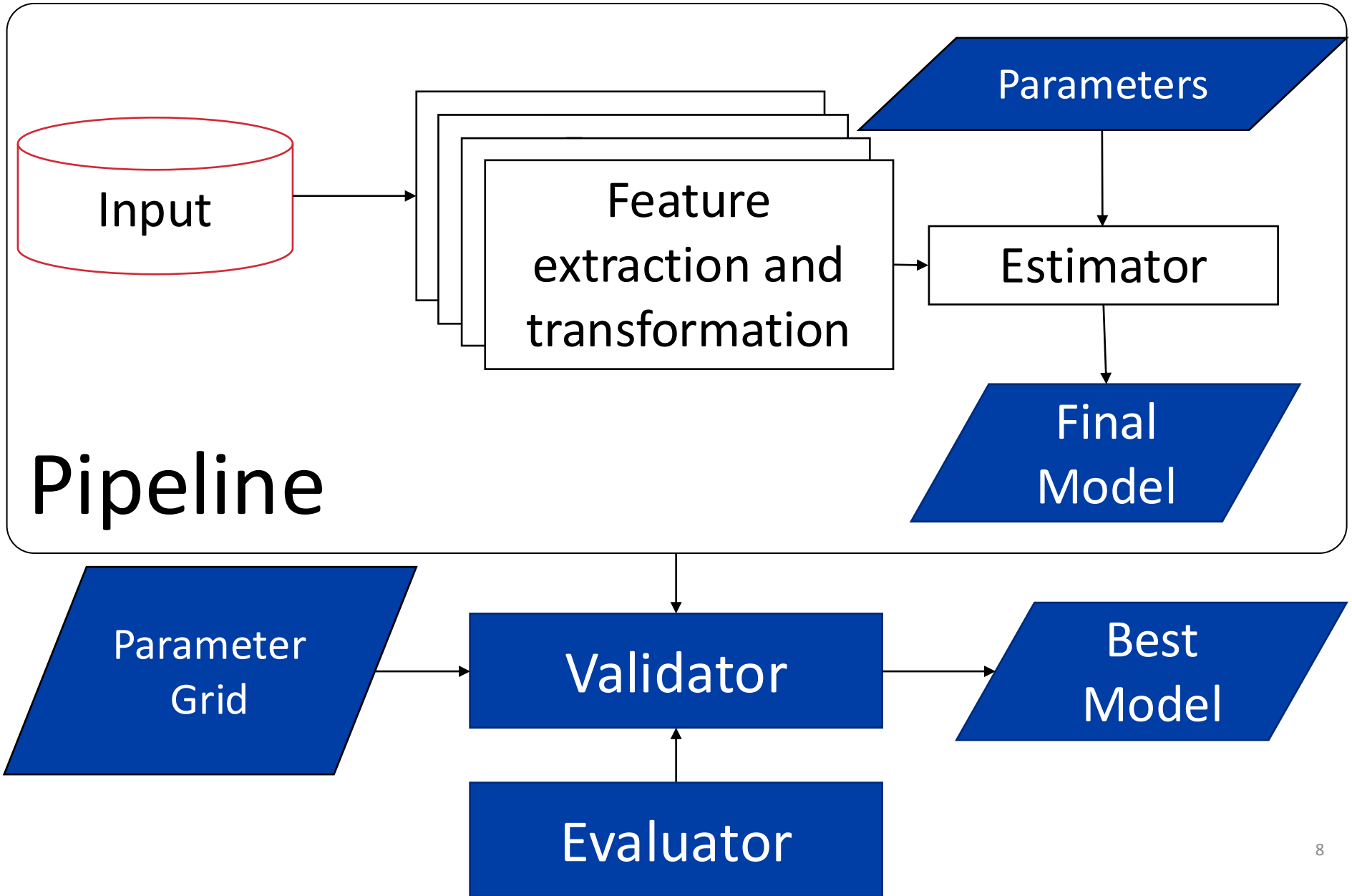
$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & & & \\ \vdots & & \ddots & \\ a_{m1} & & & a_{mn} \end{bmatrix}$$

- Dataframe.randomSplit
  - Randomly splits an input dataset
  - Helps in building training and test sets

Basic Statistics

- Column statistics
  - Minimum, Maximum, count, … etc.
- Correlation
  - Pearson's and Spearman's correlation
- Hypothesis testing
  - Chi-square Test χ^2

ML Pipeline


Parameters
Input
Feature extraction and transformation
Estimator
Final Model
Pipeline
Parameter Grid
Validator
Best Model
Evaluator

How big-data can help with ML?

- Data preparation and cleanup
  - This step is often the most time-consuming step in ML even if it does not involve training or testing
- Parallel pipeline
  - If the training data is big, Spark can run each step of the pipeline using RDD and Dataframe operations
- Parallel validation
  - Even if your data is small, you might want to train several models in parallel and choose the best

Transformations

- Used in feature extraction, dimensionality reduction, or schema transformation
- Text transformations
- Encoding
- Normalization
- Hashing
- Not to be confused with RDD transformation

TF-IDF

- Term Frequency-Inverse Document Frequency
- A measure of the importance of a term in a document
- TF: Count of a term in a document
- DF: Number of documents that contain a term
- $IDF(t, D) = \log \frac{|D|+1}{DF(t,D)+1}$
- $TFIDF(t, D) = TF(t, d) \cdot IDF(t, D)$
- Classes: HashingTF, CountVectorizer

Word2Vec

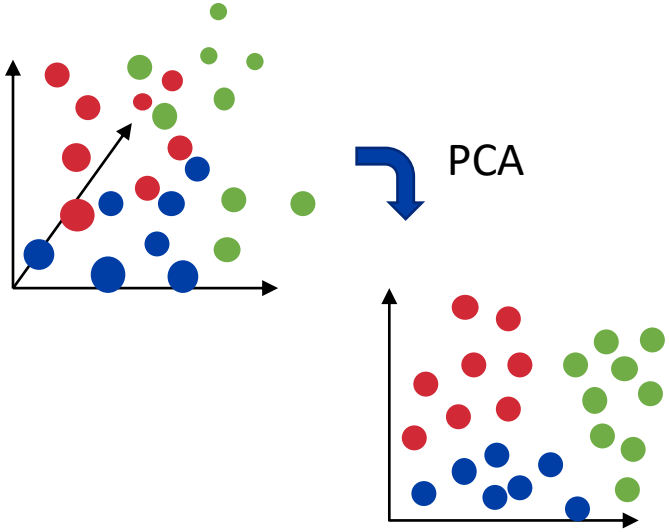
- Converts each word to a fixed-size vector
- Distance between words indicate the similarity of these words

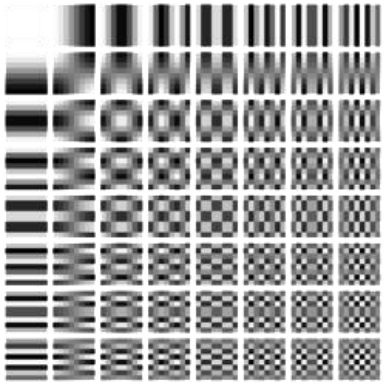
Other Text Transformers

- Tokenizer: Extracts words (tokens) from text
- StopWordRemover: Removes common words, e.g., a, the, an
- *n*-gram: Given a sequence of words, it generates subsequences of length n
- StringIndexer: Converts each unique string, e.g., label or class, to a numeric value
- IndexToString: Converts each integer value to a String value using a lookup table

Encoders

- Principal Component Analysis (PCA)
  - Reduces number of dimensions to a set of uncorrelated dimensions (components)
- DiscreteCosineTransform (DCT)
  - Frequency analysis
- OneHotEncoder: Converts categorical values to a vector with one bit set for the category





DCT

Numeric Transformers

- Binarizer: Converts numerical values to (0/1) based on a threshold
- Bucketizer: Converts continuous values to a set of n+1 buckets based on n thresholds
- QuantileDiscretizer: Places numeric values into buckets based on quantiles
- Normalizer: normalizes each vector to have unit norm. For example,

$$[4.0 \quad 10.0 \quad 2.0]$$
$$\rightarrow [0.25 \quad 0.625 \quad 0.125]$$

- MinMaxScaler: Scales each feature in a vector to a standard scale, e.g., [0.0, 1.0]

Other Transformers

- Imputer: Replaces missing values by a number or the mean
- VectorAssembler: Combines multiple attributes into a vector attribute
- VectorSlicer: Extracts a subarray of a long vector
- SQLTransformer: Applies an SQL query on the input dataset

Applying Transformers

- Simple transformers
  - Can be applied by looking at each individual record
  - E.g., `Bucketizer`, or `VectorAssembler`
  - Applied by calling the `transform` method
  - E.g., `outdf = model.transform(indf)`
- Holistic transformers
  - Need to see the entire dataset first before they can work
  - e.g., `MinMaxScaler`, `HashingTF`, `StringIndexer`
  - To apply them, you need to call `fit` then `transform`
  - e.g., `outdf = model.fit(indf).transform(indf)`

Estimators

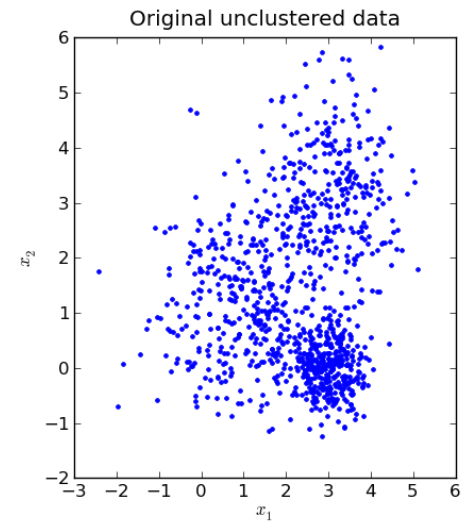
- An estimator is a machine learning algorithm that fits a model to the data
- Classification
  - Classifies data points into discrete points (categories)
- Regression
  - Estimates a continuous numeric value
- Clustering
  - Groups similar records together into clusters
- Collaborative filtering (Recommendation)
  - Predicts (missing) user ratings for items
- Frequent Pattern Mining

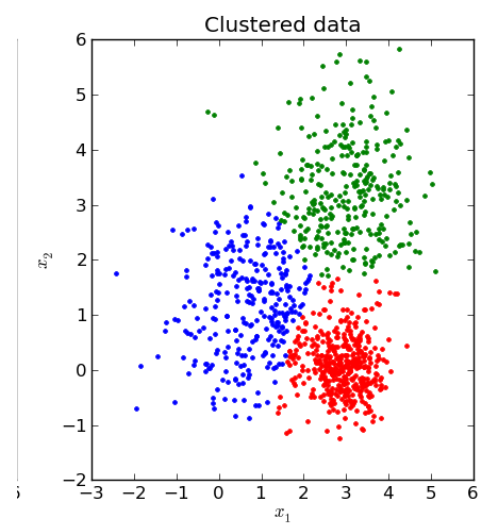
Classification and regression

- Supervised learning algorithms
- Classification
  - Logistic regression
  - Decision tree
  - Naïve Bayes
  - …
- Regression
  - Linear regression
  - Decision tree regression
  - Random forest regression
  - …

Clustering

- Unsupervised learning method
- K-means clustering. Clustering based on distance between vectors
- Latent Dirichlet allocation (LDA) Groups vectors based on some latent (hidden) variables
- Bisecting k-means. Hierarchical clustering
- Gaussian Mixture Model (GMM). Breaks down data distribution into multiple Gaussian distributions





Evaluators

- An Evaluator takes a model and produces numeric values that measure the goodness of the model for a specific dataset
- BinaryClassificationEvaluator evaluates binary classifiers using precision, recall, F-measure, area under ROC curve, … etc.
- MulticlassClassificationEvaluator evaluates multiclass classifiers using confusion matrix, accuracy, precision, recall … etc.

Evaluators

- ClusteringEvaluator evaluates clustering algorithms using sum of squared distances
- RegressionEvaluator evaluates regression models using Mean Squared Error (MSE), Root Mean Squared Error (RMSE) … etc.

Validators

- Each model has its own parameters that are usually no intuitive to tune
- A validator takes a pipeline, an evaluator, and a set of parameters and it tries all possible combinations of parameters to find the best model, i.e., the model that gives the best numeric evaluation metric
- Examples, CrossValidator and TrainValidationSplit

MOVIECLIPS.COM

Code Example on MLlib

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

Input Data

House ID	Bedrooms	Area (sqft)	…	Price
1	2	1,200		$200,000
2	3	3,200		$350,000
…				

- Goal: Build a model that estimates the price given the house features, e.g., # of bedrooms and area

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Initialization

- Similar to SparkSQL

```
val spark = SparkSession
  .builder()
  .appName(”SparkSQL Demo")
  .config(conf)
  .getOrCreate()

// Read the input
val input = spark.read
  .option("header", true)
  .option("inferSchema", true)
  .csv(inputfile)
```


UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Transformations

```
// Create a feature vector
val vectorAssembler = new VectorAssembler()
  .setInputCols(Array("bedrooms", "area"))
  .setOutputCol("features")

val linearRegression = new LinearRegression()
  .setFeaturesCol("features")
  .setLabelCol("price")
  .setMaxIter(1000)
```

Create a Pipeline

```
val pipeline = new Pipeline()
  .setStages(Array(vectorAssembler, linearRegression))

// Hyper parameter tuning
val paramGrid = new ParamGridBuilder()
  .addGrid(linearRegression.regParam,
      Array(0.3, 0.1, 0.01))
  .addGrid(linearRegression.elasticNetParam,
      Array(0.0, 0.3, 0.8, 1.0))
  .build()
```

Cross Validation

```
val crossValidator = new CrossValidator()
  .setEstimator(pipeline)
  .setEvaluator(new
RegressionEvaluator().setLabelCol("price"))
  .setEstimatorParamMaps(paramGrid)
  .setNumFolds(5)
  .setParallelism(2)

val Array(trainingData, testData) =
input.randomSplit(Array(0.8, 0.2))
val model = crossValidator.fit(trainingData)
```

Apply the model on test data

```
val predictions = model.transform(testData)
// Print the first few predictions
predictions.select("price", "prediction").show(5)

val rmse = new RegressionEvaluator()
  .setLabelCol("price")
  .setPredictionCol("prediction")
  .setMetricName("rmse")
  .evaluate(predictions)
println(s"RMSE on test set is $rmse")
```

Further Reading

- Documentation
  - http://spark.apache.org/docs/latest/ml-guide.html
- MLlib paper
  - X. Meng et al, “MLlib: Machine Learning in Apache Spark”, Journal of Machine Learning Research 17:34:1-34:7 (2016)